Curso: 0796 — Lenguajes Formales y de Programación, sección B+
Auxiliar: Carlos Samuel Aguilar Acosta
Estudiante: Diego Alejandro Barrios — carné 201900158
Fecha de entrega: 13 de octubre de 2026

Tarea 4

Gramática libre de contexto de MarkdownCheck

Contenido

1. Tabla de terminales
2. Gramática en notación BNF
3. Ajustes para análisis descendente recursivo
4. Conjuntos PRIMERO y SIGUIENTE
5. Árboles de derivación
6. Cadena inválida
7. Relación con la implementación

1. Tabla de terminales

En la columna **Origen**, "Tabla 4.5" significa que el token se usa tal como lo define el enunciado; "Modificado" y "Agregado" se justifican en la sección 1.1.

Token	Descripción	Ejemplo de lexema	Origen
NUMERAL	De 1 a 6 # al inicio de línea, seguidos de un espacio. Su cantidad es el nivel del encabezado.	##	Tabla 4.5
VIÑETA	-, * o + al inicio de línea (después de la sangría), seguido de un espacio.	-	Tabla 4.5
INDENT	Token virtual: la sangría de un ítem sube **un** nivel (2 espacios).	(sin lexema)	Modificado
DEDENT	Token virtual: la lista baja **un** nivel.	(sin lexema)	Modificado
CITA	Uno o más > al inicio de línea. En la gramática se escribe CITA[k], donde *k* es la cantidad de >.	>>, > >	Modificado
CERCA_CODIGO	Tres acentos graves al inicio de línea; abre o cierra un bloque de código.	```	Tabla 4.5
LENGUAJE_CODIGO	Lo que sigue a la cerca de apertura en la misma línea.	python	Agregado
TEXTO_LITERAL	Contenido que no se tokeniza: una línea de un bloque de código o el interior de un código en línea.	x = 2 ** 3	Agregado
REGLA_HORIZONTAL	Tres o más guiones solos en una línea.	---	Tabla 4.5
NEGRITA	Doble asterisco; abre o cierra negrita. Se reconoce antes que CURSIVA.	**	Tabla 4.5
CURSIVA	Asterisco simple fuera de posición de viñeta; abre o cierra cursiva.	*	Tabla 4.5
CODIGO_INLINE	Acento grave simple; abre o cierra código en línea.	`	Tabla 4.5
TEXTO	Texto plano: todo lo que no es un delimitador.	Notas	Tabla 4.5
SALTO_PARRAFO	Una línea vacía o de solo espacios.	\n (línea vacía)	Tabla 4.5
SALTO_LINEA	Fin de una línea que sí tiene contenido.	\n	Agregado
FIN	Fin del archivo.	(sin lexema)	Agregado

1.1 Justificación de los tokens agregados y modificados

- **SALTO_LINEA (agregado).** La tabla 4.5 no tiene un token de fin de línea, y sin él el parser no puede saber dónde termina un encabezado, un ítem o una línea de cita. Por ejemplo, # Título seguido de Texto en la línea siguiente llegaría como NUMERAL TEXTO TEXTO, y

<en_linea> del encabezado se tragaría la línea siguiente (ajuste A-1). También es lo que permite reconocer párrafos de varias líneas.

- **FIN (agregado).** Es la marca de fin de entrada (el $ del análisis LL(1)). Le permite a <documento> saber cuándo terminar y hace que SIGUIENTE(<elementos>) sea { FIN }, sin conflicto con ningún bloque.
- **TEXTO_LITERAL (agregado).** La gramática de referencia 4.6 lo usa en <bloque_codigo> y <codigo_inline>, pero la tabla 4.5 no lo declara. Representa el contenido que el léxico no tokeniza (modo literal).
- **LENGUAJE_CODIGO (agregado).** En ```python el nombre del lenguaje no es contenido del bloque: se necesita aparte para generar <code class="language-python">. Si fuera un TEXTO_LITERAL no se podría distinguir de la primera línea de código.
- **INDENT y DEDENT (modificados).** Se emite **un token por cada nivel** que sube o baja. Una sangría que pasa de 0 a 4 espacios produce dos INDENT seguidos, y la gramática los rechaza porque después de un INDENT exige una VIÑETA (sección 6). Así el error de indentación es sintáctico, como lo clasifica la sección 4.9 del enunciado.
- **CITA (modificado).** El lexema puede incluir espacios entre los > (> > equivale a >>, sección 4.5). El nivel *k* es la cantidad de > y la gramática lo usa: CITA[1] es >, CITA[2] es >> o > >, etc. Para el análisis, CITA[1], CITA[2], … se comportan como terminales distintos (sección 4.3).

El analizador léxico garantiza además dos propiedades que la gramática aprovecha: cada NEGRITA, CURSIVA y CODIGO_INLINE tiene su pareja en la misma línea (un delimitador sin cerrar se reporta como error léxico y se entrega como TEXTO), y toda línea sin contenido llega como SALTO_PARRAFO, nunca como SALTO_LINEA solo.

2. Gramática en notación BNF

Convenciones. Los no terminales van entre < > y los tokens en MAYÚSCULAS. ε es la cadena vacía. **El símbolo inicial es <documento>.**

Citas. <cita(n)>, <resto_cita(n)> y <elemento_cita(n)> son esquemas con un parámetro $n \geq 1$ (el nivel de anidamiento), y CITA[n] es un token CITA con n símbolos >. Como un archivo tiene un nivel máximo de anidamiento N, el esquema equivale a escribir esas tres reglas para $n = 1, 2, \ldots, N$: sigue siendo una gramática libre de contexto ordinaria. En el nivel N la regla <elemento_cita(N)> solo tiene su primera alternativa.

```
<documento>        ::= <elementos> FIN

<elementos>        ::= <elemento_doc> <elementos>
                     | ε

<elemento_doc>     ::= <bloque>
                     | SALTO_PARRAFO

<bloque>           ::= <encabezado>
                     | <parrafo>
                     | <lista>
                     | <cita(1)>
                     | <bloque_codigo>
                     | <regla_horizontal>

<encabezado>       ::= NUMERAL <en_linea> SALTO_LINEA

<parrafo>          ::= <linea_texto> <resto_parrafo>
<resto_parrafo>    ::= <parrafo>
                     | ε
<linea_texto>      ::= <elemento_linea> <en_linea> SALTO_LINEA

<lista>            ::= <item_lista> <resto_lista>
<resto_lista>      ::= <lista>
                     | ε
<item_lista>       ::= VIÑETA <en_linea> SALTO_LINEA <sublista_opc>
<sublista_opc>     ::= <sublista>
                     | ε
<sublista>         ::= INDENT <lista> DEDENT

<cita(n)>          ::= <elemento_cita(n)> <resto_cita(n)>
<resto_cita(n)>    ::= <cita(n)>
                     | ε
<elemento_cita(n)> ::= CITA[n] <contenido_cita>
                     | <cita(n+1)>
<contenido_cita>   ::= <encabezado>
                     | <en_linea> SALTO_LINEA
```

```
<bloque_codigo>     ::= CERCA_CODIGO <lenguaje_opc> SALTO_LINEA
                        <contenido_codigo> CERCA_CODIGO SALTO_LINEA
<lenguaje_opc>      ::= LENGUAJE_CODIGO
                    | ε
<contenido_codigo>  ::= TEXTO_LITERAL SALTO_LINEA <contenido_codigo>
                    | ε

<regla_horizontal>  ::= REGLA_HORIZONTAL SALTO_LINEA

<en_linea>          ::= <elemento_linea> <en_linea>
                    | ε
<elemento_linea>    ::= <negrita>
                    | <cursiva>
                    | <codigo_inline>
                    | TEXTO

<negrita>           ::= NEGRITA <en_linea_n> NEGRITA
<en_linea_n>        ::= <elemento_n> <en_linea_n>
                    | ε
<elemento_n>        ::= <cursiva>
                    | <codigo_inline>
                    | TEXTO

<cursiva>           ::= CURSIVA <en_linea_c> CURSIVA
<en_linea_c>        ::= <elemento_c> <en_linea_c>
                    | ε
<elemento_c>        ::= <negrita>
                    | <codigo_inline>
                    | TEXTO

<codigo_inline>     ::= CODIGO_INLINE <literal_opc> CODIGO_INLINE
<literal_opc>       ::= TEXTO_LITERAL
                    | ε
```

Son 31 no terminales (contando cada esquema de cita una vez).

2.1 Cobertura del alcance obligatorio

Elemento del alcance	Reglas que lo reconocen
Encabezados # a ######	<encabezado>; el nivel lo da el lexema de NUMERAL.
Párrafos de una o más líneas	<parrafo>, <resto_parrafo>, <linea_texto>
Negrita y cursiva, también combinadas	<negrita>, <cursiva>, <en_linea_n>, <en_linea_c> (recursión mutua)
Código en línea	<codigo_inline>, <literal_opc>
Bloques de código, con lenguaje opcional	<bloque_codigo>, <lenguaje_opc>, <contenido_codigo>

Elemento del alcance	Reglas que lo reconocen
Listas no ordenadas, 3 o más niveles	<lista> → <item_lista> → <sublista> → <lista>: cada INDENT abre un nivel más, sin límite.
Citas anidadas (> y >>)	<cita(n)> → <elemento_cita(n)> → <cita(n+1)>, sin límite de niveles.
Regla horizontal	<regla_horizontal>
Líneas en blanco entre bloques	<elemento_doc> → SALTO_PARRAFO

3. Ajustes para análisis descendente recursivo

Cada ajuste indica la regla de la gramática de referencia 4.6, el problema que tiene para un parser que decide viendo un solo token, y el cambio realizado.

A-1. Fin de línea en las reglas de línea

Antes (4.6): <encabezado> ::= NUMERAL <en_linea> (lo mismo en <item_lista>, <cita> y <regla_horizontal>).

Problema. <en_linea> es una repetición que termina cuando el siguiente token no puede empezar un elemento en línea. Para # Título seguido de una línea Texto, la entrada es NUMERAL TEXTO TEXTO: el TEXTO de la segunda línea está en PRIMERO(<elemento_linea>) y también en SIGUIENTE(<en_linea>), porque un párrafo puede seguir al encabezado. El parser no puede decidir si ese TEXTO sigue en el encabezado o empieza un párrafo, y la gramática es ambigua: hay dos árboles para la misma entrada.

Ajuste. Se agregó el token SALTO_LINEA al final de <encabezado>, <linea_texto>, <item_lista>, <contenido_cita>, <regla_horizontal> y de cada línea de <bloque_codigo>. Ahora SIGUIENTE(<en_linea>) = { SALTO_LINEA }, que no puede empezar un elemento en línea.

A-2. Las líneas vacías pasan a <elemento_doc>

Antes (4.6): <documento> ::= <bloque> <documento> | ε y <parrafo> ::= <en_linea> SALTO_PARRAFO.

Problema. SALTO_PARRAFO solo aparece al final de un párrafo, y como <en_linea> puede ser ε, cada línea vacía que sigue a un encabezado, una lista, un bloque de código o una regla horizontal se reconoce como un **párrafo vacío**. En el ejemplo de la sección 4.7 del enunciado eso pasa cinco veces, mientras que su Reporte 2 cuenta solo 2 párrafos. Además, el último párrafo (Fin del documento.) se rechaza, porque el archivo termina sin una línea vacía después de él.

Ajuste.

<documento> ::= <elementos> FIN
<elementos> ::= <elemento_doc> <elementos> | ε
<elemento_doc> ::= <bloque> | SALTO_PARRAFO

Una línea vacía es un elemento del documento por sí misma, en cualquier lugar y en cualquier cantidad. Además, FIN marca el final de la entrada.

A-3. Párrafos de una o más líneas, nunca vacíos

Antes (4.6): <parrafo> ::= <en_linea> SALTO_PARRAFO.

Problema. Además de lo visto en A-2, un párrafo no puede tener varias líneas una vez que existe SALTO_LINEA (A-1), y <en_linea> vacío permite un párrafo sin contenido.

Ajuste.

<parrafo> ::= <linea_texto> <resto_parrafo>
<resto_parrafo> ::= <parrafo> | ε
<linea_texto> ::= <elemento_linea> <en_linea> SALTO_LINEA

<linea_texto> empieza con un <elemento_linea> obligatorio, así que PRIMERO(<parrafo>) = { TEXTO, NEGRITA, CURSIVA, CODIGO_INLINE } no contiene ε ni SALTO_LINEA. La recursión es por la derecha.

A-4. Factorización de <lista>

Antes (4.6): <lista> ::= <item_lista> <lista> | <item_lista>.

Problema. Las dos alternativas empiezan con <item_lista>, que empieza con VIÑETA: viendo una VIÑETA el parser no puede elegir entre ellas.

Ajuste (factorización por la izquierda).

<lista> ::= <item_lista> <resto_lista>
<resto_lista> ::= <lista> | ε

A-5. Factorización de <item_lista>

Antes (4.6): <item_lista> ::= VIÑETA <en_linea> <sublista> | VIÑETA <en_linea>.

Problema. Prefijo común VIÑETA <en_linea>.

Ajuste. Se factoriza la parte común y la sublista queda opcional; además lleva el SALTO_LINEA de A-1:

<item_lista> ::= VIÑETA <en_linea> SALTO_LINEA <sublista_opc>
<sublista_opc> ::= <sublista> | ε

La decisión se toma con un solo token: si después del SALTO_LINEA viene INDENT, hay sublista.

A-6. Citas: factorización y nivel de anidamiento

Antes (4.6): <cita> ::= CITA <bloque> <cita> | CITA <bloque>.

Problemas.

1. Las dos alternativas comparten el prefijo CITA <bloque>.
2. La regla no usa el nivel. Para > a y >> b deriva dos líneas hermanas de la misma cita, así que el árbol no tiene el anidamiento y el Reporte 1 no podría generar un <blockquote> dentro de otro.
3. El contenido de cada línea es un <bloque>. Eso admite listas y bloques de código dentro de una cita, fuera del alcance definido. Además, una línea de texto solo puede ser un <parrafo>, que exige un SALTO_PARRAFO, así que dos líneas de cita seguidas, como las del ejemplo 4.7, no se pueden derivar.

Ajuste.

<cita(n)> ::= <elemento_cita(n)> <resto_cita(n)>
<resto_cita(n)> ::= <cita(n)> | ε
<elemento_cita(n)> ::= CITA[n] <contenido_cita> | <cita(n+1)>
<contenido_cita> ::= <encabezado> | <en_linea> SALTO_LINEA

<resto_cita(n)> factoriza el prefijo. <elemento_cita(n)> decide con el nivel del token: una CITA[n] es una línea de esta cita; una CITA de nivel mayor abre la cita interna <cita(n+1)>. Si se salta del nivel 1 al 3, la cita de nivel 2 queda sin líneas propias, igual que en Markdown. El contenido se limita a un encabezado o a una línea de texto con formato.

A-7. Bloques de código con lenguaje y varias líneas

Antes (4.6): <bloque_codigo> ::= CERCA_CODIGO TEXTO_LITERAL CERCA_CODIGO.

Problema. No admite el lenguaje (```python), ni un bloque vacío, ni separa las líneas del contenido.

Ajuste.

<bloque_codigo> ::= CERCA_CODIGO <lenguaje_opc> SALTO_LINEA
<contenido_codigo> CERCA_CODIGO SALTO_LINEA
<lenguaje_opc> ::= LENGUAJE_CODIGO | ε
<contenido_codigo> ::= TEXTO_LITERAL SALTO_LINEA <contenido_codigo> | ε

A-8. Negrita y cursiva: tres versiones de <en_linea>

Antes (4.6): <negrita> ::= "**" <en_linea> "**" y <cursiva> ::= "*" <en_linea> "*".

Problema (conflicto LL(1)). Dentro de una negrita, <en_linea> puede empezar con otra <negrita>, así que NEGRITA ∈ PRIMERO(<en_linea>). Pero <en_linea> va seguido del ** de cierre, así que también NEGRITA ∈ SIGUIENTE(<en_linea>). Al ver ** el parser no sabe si abre una negrita anidada o cierra la actual. Lo mismo pasa con la cursiva y *.

Ajuste. Dentro de una negrita se usa <en_linea_n>, que admite todo menos otra negrita directa; dentro de una cursiva, <en_linea_c>, que admite todo menos otra cursiva directa:

<negrita> ::= NEGRITA <en_linea_n> NEGRITA
<en_linea_n> ::= <elemento_n> <en_linea_n> | ε
<elemento_n> ::= <cursiva> | <codigo_inline> | TEXTO

<cursiva> ::= CURSIVA <en_linea_c> CURSIVA
<en_linea_c> ::= <elemento_c> <en_linea_c> | ε
<elemento_c> ::= <negrita> | <codigo_inline> | TEXTO

Ahora PRIMERO(<en_linea_n>) no contiene NEGRITA y PRIMERO(<en_linea_c>) no contiene CURSIVA: el siguiente delimitador igual siempre **cierra**. Se conserva la recursión mutua que pide el enunciado: *a **b *c* d** e* tiene cursiva, negrita y cursiva anidadas.

A-9. Código en línea vacío

Antes (4.6): <codigo_inline> ::= "” TEXTO_LITERAL “".

Ajuste. <codigo_inline> ::= CODIGO_INLINE <literal_opc> CODIGO_INLINE con <literal_opc> ::= TEXTO_LITERAL | ε, para aceptar ``.

A-10. Notación uniforme

La 4.6 mezcla nombres de token con literales ("**", "*", "`"). Se usaron solo los nombres de token en MAYÚSCULAS de la tabla de terminales, y se declaró el símbolo inicial.

3.1 Verificación: sin recursión por la izquierda

La gramática de referencia 4.6 no tenía recursión por la izquierda, y ningún ajuste la introduce. Se comprobó siguiendo, desde cada no terminal, la cadena de primeros símbolos de sus producciones (incluidos los que vienen después de un no terminal anulable). Ninguna cadena vuelve al no terminal de partida:

- Todas las recursiones son **por la derecha** (<elementos>, <resto_parrafo>, <resto_lista>, <resto_cita(n)>, <contenido_codigo>, <en_linea>, <en_linea_n>, <en_linea_c>) o están **después de un terminal** (<sublista> después de INDENT, <negrita> después de CURSIVA, <cursiva> después de NEGRITA).
- La cadena <cita(n)> ⇒ <elemento_cita(n)> ⇒ <cita(n+1)> no es recursión por la izquierda: llega a un no terminal **distinto** (el nivel aumenta) y termina cuando *n* alcanza el nivel del token, donde se consume CITA[n].

3.2 Verificación: alternativas factorizadas

Ningún no terminal tiene dos alternativas que empiecen con el mismo símbolo. Las alternativas que podían empezar con el mismo token en la 4.6 (<lista>, <item_lista>, <cita>, <negrita>, <cursiva>) quedaron resueltas con los ajustes A-4 a A-8. Los únicos conflictos que quedan son los de la sección 4.3, entre continuar o terminar una repetición.

4. Conjuntos PRIMERO y SIGUIENTE

4.1 Abreviaturas

Para que la tabla se pueda leer, se definen cuatro conjuntos que se repiten:

Nombre	Contenido
E (inicio de elemento en línea)	{ TEXTO, NEGRITA, CURSIVA, CODIGO_INLINE }
C≥n (citas de nivel *n* o mayor)	{ CITA[k] : k ≥ n }. **C≥1** son todas las citas.
B (inicio de bloque)	**E** ∪ **C≥1** ∪ { NUMERAL, VIÑETA, CERCA_CODIGO, REGLA_HORIZONTAL }
S (lo que puede seguir a un bloque)	**B** ∪ { SALTO_PARRAFO, FIN }

4.2 Tabla

$ es el fin de la entrada, que solo sigue al símbolo inicial.

No terminal	PRIMERO	SIGUIENTE
<documento>	**B** ∪ { SALTO_PARRAFO, FIN }	{ $ }
<elementos>	**B** ∪ { SALTO_PARRAFO, ε }	{ FIN }
<elemento_doc>	**B** ∪ { SALTO_PARRAFO }	**S**
<bloque>	**B**	**S**
<encabezado>	{ NUMERAL }	**S**
<parrafo>	**E**	**S**
<resto_parrafo>	**E** ∪ { ε }	**S**
<linea_texto>	**E**	**S**
<lista>	{ VIÑETA }	**S** ∪ { DEDENT }
<resto_lista>	{ VIÑETA, ε }	**S** ∪ { DEDENT }
<item_lista>	{ VIÑETA }	**S** ∪ { DEDENT }
<sublista_opc>	{ INDENT, ε }	**S** ∪ { DEDENT }
<sublista>	{ INDENT }	**S** ∪ { DEDENT }
<cita(n)>	**C≥n**	**S**
<resto_cita(n)>	**C≥n** ∪ { ε }	**S**
<elemento_cita(n)>	**C≥n**	**S**
<contenido_cita>	**E** ∪ { NUMERAL, SALTO_LINEA }	**S**
<bloque_codigo>	{ CERCA_CODIGO }	**S**
<lenguaje_opc>	{ LENGUAJE_CODIGO, ε }	{ SALTO_LINEA }
<contenido_codigo>	{ TEXTO_LITERAL, ε }	{ CERCA_CODIGO }
<regla_horizontal>	{ REGLA_HORIZONTAL }	**S**
<en_linea>	**E** ∪ { ε }	{ SALTO_LINEA }

No terminal	PRIMERO	SIGUIENTE
<elemento_linea>	**E**	**E** ∪ { SALTO_LINEA }
<negrita>	{ NEGRITA }	**E** ∪ { SALTO_LINEA }
<en_linea_n>	{ TEXTO, CURSIVA, CODIGO_INLINE, ε }	{ NEGRITA }
<elemento_n>	{ TEXTO, CURSIVA, CODIGO_INLINE }	**E**
<cursiva>	{ CURSIVA }	**E** ∪ { SALTO_LINEA }
<en_linea_c>	{ TEXTO, NEGRITA, CODIGO_INLINE, ε }	{ CURSIVA }
<elemento_c>	{ TEXTO, NEGRITA, CODIGO_INLINE }	**E**
<codigo_inline>	{ CODIGO_INLINE }	**E** ∪ { SALTO_LINEA }
<literal_opc>	{ TEXTO_LITERAL, ε }	{ CODIGO_INLINE }

Algunos resultados que conviene explicar:

- **SIGUIENTE(<lista>) incluye DEDENT** porque una lista puede estar dentro de <sublista> ::= INDENT <lista> DEDENT.
- **SIGUIENTE(<cita(n)>) = S para todo *n*.** Una cita interna termina en el mismo punto que la de afuera, o justo antes de una línea de nivel menor (CITA[k], que ya está en **S**).
- **SIGUIENTE(<elemento_n>) = E** y no incluye SALTO_LINEA: dentro de una negrita, lo que sigue a un elemento es otro elemento o el NEGRITA de cierre, y el léxico garantiza que la negrita cierra en la misma línea.

4.3 Conflictos y su resolución

Un no terminal con una alternativa ε cumple la condición LL(1) si PRIMERO(alternativa no vacía) ∩ SIGUIENTE(no terminal) = ∅. Con la tabla anterior, todas las alternativas tienen conjuntos PRIMERO disjuntos y quedan **tres conflictos**, todos del mismo tipo: decidir entre **continuar** una repetición o **terminarla**.

No terminal	Alternativas en conflicto	Tokens en conflicto	Por qué ocurre
<resto_parrafo>	<parrafo> / ε	**E** (4 tokens)	Una línea de texto puede ser la siguiente del mismo párrafo o el inicio de otro párrafo: **E** ⊂ **S**.
<resto_lista>	<lista> / ε	VIÑETA	Una viñeta puede ser el siguiente ítem de la misma lista o el inicio de otra lista.
<resto_cita(n)>	<cita(n)> / ε	**C≥n**	Una línea de cita de nivel ≥ *n* puede seguir en la misma cita o empezar otra cita.

En la tabla de análisis LL(1) calculada para citas de hasta 3 niveles, estos tres conflictos ocupan 11 de las 194 celdas definidas: 4 de <resto_parrafo>, 1 de <resto_lista> y 3 + 2 + 1 de <resto_cita(1)>, <resto_cita(2)> y <resto_cita(3)>.

Origen. Estos conflictos vienen de una ambigüedad real de la gramática: <elementos> permite poner dos bloques del mismo tipo uno junto al otro. Las líneas uno y dos sin línea vacía entre ellas tienen dos derivaciones: un párrafo de dos líneas o dos párrafos de una línea. Pasa lo mismo con dos ítems seguidos y con dos líneas de cita seguidas.

Regla de desambiguación: coincidencia más larga. Ante un conflicto, el parser elige siempre **continuar** (la alternativa no vacía): mientras la siguiente línea pueda pertenecer al párrafo, la lista o la cita actual, la agrega. Es la misma solución que se usa para el *else* colgante. Coincide con el significado de Markdown: las líneas consecutivas forman un mismo párrafo, una misma lista o una misma cita, y para separarlas hace falta una línea vacía (SALTO_PARRAFO, que no está en ningún PRIMERO de estas repeticiones).

En el caso de las citas, la regla compara el nivel del token: si llega CITA[k] con $k \geq n$, la cita de nivel n continúa; si $k < n$, la línea pertenece a una cita de afuera y <resto_cita(n)> toma ε.

El nivel de las citas no es un conflicto. Si CITA se tratara como un solo terminal, <elemento_cita(n)> tendría conflicto entre CITA[n] <contenido_cita> y <cita(n+1)>, porque ambas empiezan con CITA. El parser lo resuelve con el atributo nivel del token (la cantidad de >), lo que equivale a tratar CITA[1], CITA[2], … como terminales distintos. Así, PRIMERO(CITA[n] …) = { CITA[n] } y PRIMERO(<cita(n+1)>) = **C≥n+1** son disjuntos.

Conflictos eliminados respecto a la 4.6: el prefijo común de <lista>, <item_lista> y <cita> (ajustes A-4 a A-6), la ambigüedad del fin de línea (A-1) y el ** / * de apertura o cierre (A-8).

5. Árboles de derivación

Los árboles siguen la BNF de la sección 2 y muestran todas las producciones, incluidas las que derivan ε (círculos grises punteados). Los no terminales van en azul, los tokens en ámbar con su lexema debajo, y la raíz <documento> en azul oscuro. Cada imagen está también en formato vectorial (.svg) en la carpeta img/, para verla sin pérdida de calidad a cualquier tamaño.

5.1 Cadena 1

```
# Notas **importantes**

- Lexico
  - AFD
    - Estados con *cursiva*
- Sintactico
```

Secuencia de tokens (27):

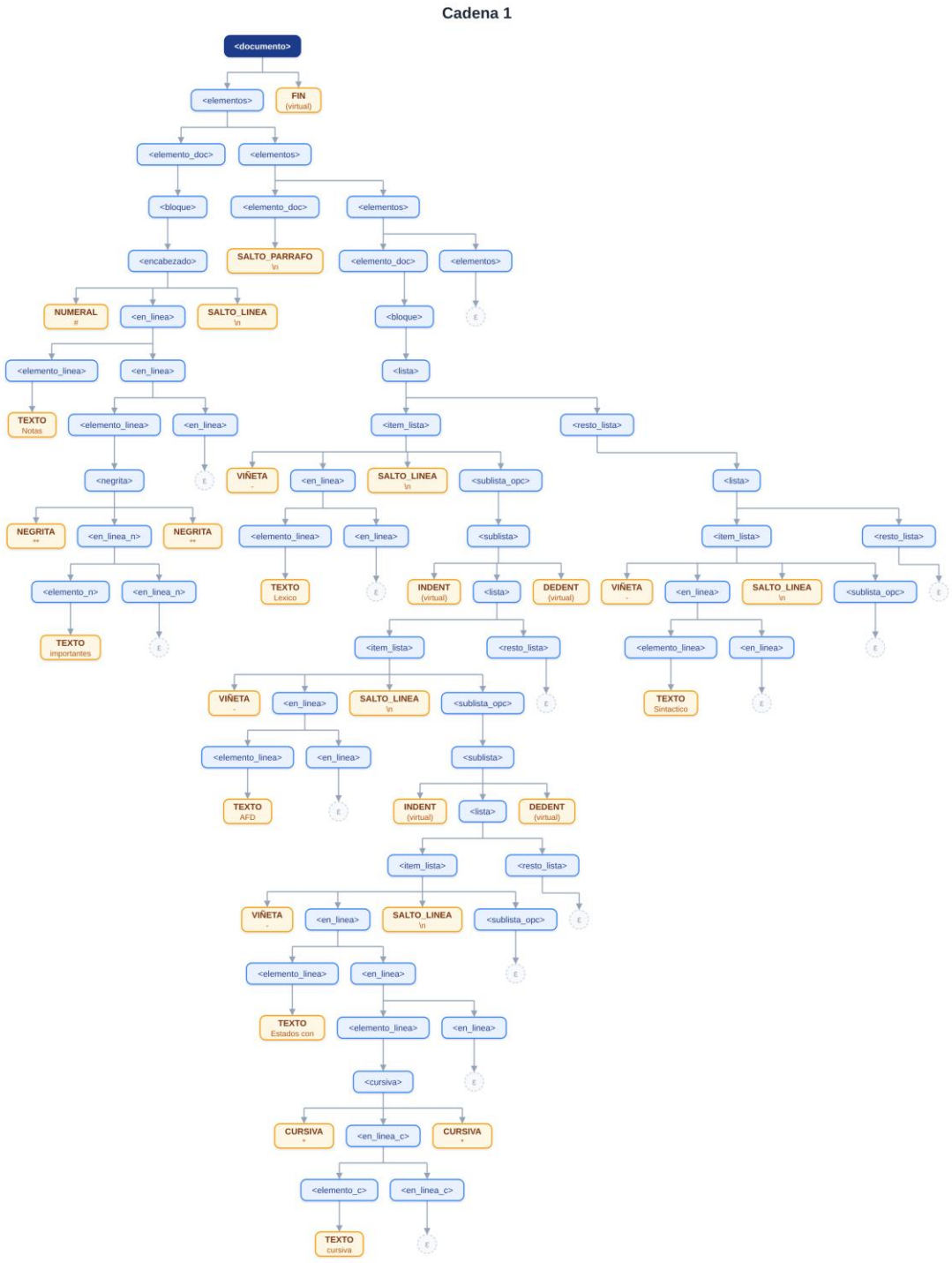
No.	Token	Lexema	Línea:columna
1	NUMERAL	#	1:1
2	TEXTO	Notas	1:3
3	NEGRITA	**	1:9
4	TEXTO	importantes	1:11
5	NEGRITA	**	1:22
6	SALTO_LINEA	\n	1:24
7	SALTO_PARRAFO	\n	2:1
8	VIÑETA	-	3:1
9	TEXTO	Lexico	3:3
10	SALTO_LINEA	\n	3:9
11	INDENT	(virtual)	4:1
12	VIÑETA	-	4:3
13	TEXTO	AFD	4:5
14	SALTO_LINEA	\n	4:8
15	INDENT	(virtual)	5:3
16	VIÑETA	-	5:5
17	TEXTO	Estados con	5:7
18	CURSIVA	*	5:19
19	TEXTO	cursiva	5:20
20	CURSIVA	*	5:27
21	SALTO_LINEA	\n	5:28
22	DEDENT	(virtual)	6:1
23	DEDENT	(virtual)	6:1

No.	Token	Lexema	Línea:columna
24	VIÑETA	-	6:1
25	TEXTO	Sintactico	6:3
26	SALTO_LINEA	\n	6:13
27	FIN	(virtual)	7:1

En una línea: NUMERAL TEXTO NEGRITA TEXTO NEGRITA SALTO_LINEA SALTO_PARRAFO VIÑETA TEXTO SALTO_LINEA INDENT VIÑETA TEXTO SALTO_LINEA INDENT VIÑETA TEXTO CURSIVA TEXTO CURSIVA SALTO_LINEA DEDENT DEDENT VIÑETA TEXTO SALTO_LINEA FIN.

Los dos DEDENT seguidos (tokens 22 y 23) cierran de una vez los niveles 3 y 2, porque Sintactico vuelve directamente al nivel 1.

Árbol de derivación:



Árbol de derivación de la cadena 1

Puntos que se leen en el árbol:

- **Tres niveles de lista**: <lista> → <item_lista> → <sublista_opc> → <sublista> (INDENT <lista> DEDENT), dos veces.
- **El regreso al nivel 1**: el ítem Sintactico cuelga del <resto_lista> de la lista de nivel 1. Las listas de nivel 3 y 2 terminan con <resto_lista> → ε, porque el token siguiente es un DEDENT y no una VIÑETA.
- **La negrita del encabezado y la cursiva del tercer nivel** cuelgan de su <elemento_linea>, cada una con su <en_linea_n> o <en_linea_c>.

5.2 Cadena 2

````
> Una cita con `codigo`
>> Una cita anidada

```python
x = 2 ** 3
```
````

Secuencia de tokens (21). En la columna Token, CITA[k] indica el nivel que usa la gramática:

No.	Token	Lexema	Línea:columna
1	CITA[1]	>	1:1
2	TEXTO	Una cita con	1:3
3	CODIGO_INLINE	`	1:16
4	TEXTO_LITERAL	codigo	1:17
5	CODIGO_INLINE	`	1:23
6	SALTO_LINEA	\n	1:24
7	CITA[2]	>>	2:1
8	TEXTO	Una cita anidada	2:4
9	SALTO_LINEA	\n	2:20
10	SALTO_PARRAFO	\n	3:1
11	CERCA_CODIGO	```	4:1
12	LENGUAJE_CODIGO	python	4:4
13	SALTO_LINEA	\n	4:10
14	TEXTO_LITERAL	x = 2 ** 3	5:1
15	SALTO_LINEA	\n	5:11
16	CERCA_CODIGO	```	6:1
17	SALTO_LINEA	\n	6:4
18	SALTO_PARRAFO	\n	7:1
19	REGLA_HORIZONTAL	---	8:1
20	SALTO_LINEA	\n	8:4
21	FIN	(virtual)	9:1

En una línea: CITA[1] TEXTO CODIGO_INLINE TEXTO_LITERAL CODIGO_INLINE SALTO_LINEA CITA[2] TEXTO SALTO_LINEA SALTO_PARRAFO CERCA_CODIGO LENGUAJE_CODIGO SALTO_LINEA TEXTO_LITERAL SALTO_LINEA CERCA_CODIGO SALTO_LINEA SALTO_PARRAFO REGLA_HORIZONTAL SALTO_LINEA FIN.

El ** de la línea 5 no es una NEGRITA: dentro del bloque de código el léxico está en modo literal, y toda la línea es un solo TEXTO_LITERAL.

Árbol de derivación:



Árbol de derivación de la cadena 2

Puntos que se leen en el árbol:

- **El anidamiento de la cita**: la línea >> no es hermana de la primera. <resto_cita(1)> continúa la cita de nivel 1 (regla de la coincidencia más larga, con $k = 2 \geq 1$), y su <elemento_cita(1)> toma la alternativa <cita(2)> porque el token es CITA[2]. Ahí se consume CITA[2].
- **El código en línea** `codigo` es un <codigo_inline> con su <literal_opc> → TEXTO_LITERAL.
- **Las líneas vacías** son <elemento_doc> → SALTO_PARRAFO (ajuste A-2), hermanos de los bloques.

6. Cadena inválida

```
- Uno
   - Salto de dos niveles
```

La segunda línea tiene 4 espacios de sangría: salta del nivel 1 al nivel 3 sin pasar por el 2.

Secuencia de tokens (11). El léxico no reporta error: emite un INDENT por cada nivel que sube (sección 1.1), así que salen **dos INDENT seguidos**.

No.	Token	Lexema	Línea:columna
1	VIÑETA	-	1:1
2	TEXTO	Uno	1:3
3	SALTO_LINEA	\n	1:6
4	INDENT	(virtual)	2:1
5	INDENT	(virtual)	2:3
6	VIÑETA	-	2:5
7	TEXTO	Salto de dos niveles	2:7
8	SALTO_LINEA	\n	2:27
9	DEDENT	(virtual)	3:1
10	DEDENT	(virtual)	3:1
11	FIN	(virtual)	3:1

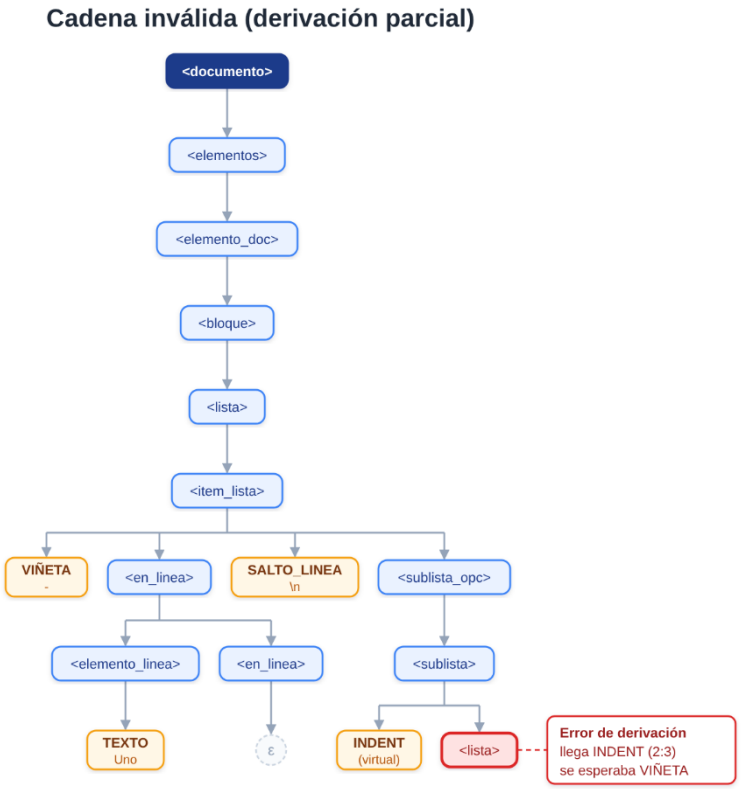
Traza del análisis predictivo. La cima de la pila está a la izquierda; la entrada muestra los siguientes cuatro tokens.

Pila	Entrada	Acción
<documento>	VIÑETA TEXTO SALTO_LINEA INDENT …	<documento> → <elementos> FIN
<elementos> FIN	VIÑETA TEXTO SALTO_LINEA INDENT …	<elementos> → <elemento_doc> <elementos>
<elemento_doc> <elementos> FIN	VIÑETA …	<elemento_doc> → <bloque>
<bloque> <elementos> FIN	VIÑETA …	<bloque> → <lista>
<lista> <elementos> FIN	VIÑETA …	<lista> → <item_lista> <resto_lista>
<item_lista> <resto_lista> …	VIÑETA …	<item_lista> → VIÑETA <en_linea> SALTO_LINEA <sublista_opc>
VIÑETA <en_linea> SALTO_LINEA …	VIÑETA TEXTO SALTO_LINEA INDENT	consumir VIÑETA
<en_linea> SALTO_LINEA …	TEXTO SALTO_LINEA INDENT INDENT	<en_linea> → <elemento_linea> <en_linea>
<elemento_linea> <en_linea> …	TEXTO …	<elemento_linea> → TEXTO

Pila	Entrada	Acción
TEXTO <en_linea> …	TEXTO …	consumir TEXTO
<en_linea> SALTO_LINEA …	SALTO_LINEA INDENT INDENT VIÑETA	<en_linea> → ε (SALTO_LINEA ∈ SIGUIENTE)
SALTO_LINEA <sublista_opc> …	SALTO_LINEA …	consumir SALTO_LINEA
<sublista_opc> <resto_lista> …	INDENT INDENT VIÑETA TEXTO	<sublista_opc> → <sublista>
<sublista> <resto_lista> …	INDENT …	<sublista> → INDENT <lista> DEDENT
INDENT <lista> DEDENT …	INDENT INDENT VIÑETA TEXTO	consumir INDENT (token 4)
<lista> DEDENT …	INDENT VIÑETA TEXTO SALTO_LINEA	**error**: la celda M[<lista>, INDENT] está vacía

Dónde y por qué falla. Después del primer INDENT, la producción <sublista> ::= INDENT <lista> DEDENT exige una <lista>, y PRIMERO(<lista>) = { VIÑETA }. <lista> no deriva ε, así que no hay ninguna producción para el token que llega: el **segundo INDENT (token 5, línea 2, columna 3)**. La gramática solo permite **un nivel por sublista**: cada INDENT tiene que ir seguido directamente de un ítem. Un salto de dos niveles deja un nivel sin ítem, y la cadena no pertenece al lenguaje.

Derivación parcial (en rojo, el no terminal que no se pudo expandir):



Derivación parcial de la cadena inválida

Error reportado (sección 4.9): INDENTACION_INVALIDA, fase **sintáctica**: “Indentación inválida en línea 2, columna 3: la sangría sube más de un nivel de una vez”. Para no detenerse en el primer error (modo pánico), el parser del proyecto analiza el nivel sobrante como otra <sublista>. Así cada INDENT encuentra su DEDENT y el resto del archivo se sigue analizando con normalidad.

7. Relación con la implementación

El parser descendente recursivo del Proyecto 2 implementa esta gramática con **una función por no terminal**. Las recursiones por la derecha de la forma <resto_x> ::= <x> | ε se implementan como ciclos while, y la regla de la coincidencia más larga es la condición del ciclo:

Regla	**En el parser**
<resto_parrafo>	while el token actual ∈ **E**: agregar otra <linea_texto>
<resto_lista>	while el token actual es VIÑETA: agregar otro <item_lista>
<resto_cita(n)>	while el token actual es CITA[k] con $k \geq n$: otra <elemento_cita(n)>
<elemento_cita(n)>	if $k > n$: llamar a parsear_cita(n + 1); si no, consumir CITA[n]
<en_linea>, <en_linea_n>, <en_linea_c>	while el token actual ∈ su PRIMERO
<sublista_opc>, <lenguaje_opc>, <literal_opc>	if el token actual ∈ su PRIMERO

Por eso el árbol que construye el programa agrupa las repeticiones: una sola <lista> con todos sus <item_lista> como hijos, en lugar de la cadena de <resto_lista> que se dibuja en la sección 5. Las dos formas describen la misma estructura.